

Catalogizer

Catalogizer

Tagline: Advanced multi-protocol media collection management — detect, catalog, and enrich everything you own.

Summary: Catalogizer is a self-hostable media collection management system that automatically detects, categorizes, and organizes media across SMB, FTP, NFS, WebDAV, and local filesystems, with real-time monitoring, encrypted storage, external metadata enrichment, and a modern React UI backed by a high-performance Go API.

Short description (~40 words): A production-grade, multi-protocol media library manager. A Go/Gin REST API detects 50+ media types across SMB/FTP/NFS/WebDAV/local sources, enriches from TMDB/IMDB/MusicBrainz/Steam and more, and serves a real-time React web app over an encrypted SQLCipher database.

Long description (150-250 words): Catalogizer solves the problem of media sprawl across heterogeneous network storage. Rather than forcing files into one location, it speaks the protocols storage already uses — SMB/CIFS, FTP/FTPS, NFS, WebDAV, and local filesystem — behind a single unified client abstraction, so sources can be mixed and swapped freely. A Go backend (Gin REST API) continuously monitors those sources, detects and classifies 50+ media types (movies, TV, music, games, software, documentaries), and enriches each item from external providers including TMDB, IMDB, TVDB, MusicBrainz, Spotify, and Steam. Results stream to a TypeScript React frontend over WebSockets for live updates, while all metadata is persisted in an encrypted SQLCipher database with JWT-based, role-based authentication. The system is engineered for resilience: temporary SMB outages are absorbed by exponential-backoff reconnection, a circuit-breaker pattern, continuous health monitoring, and an offline metadata cache that keeps serving user requests while sources are down. Beyond cataloging, Catalogizer adds analytics (growth trends, quality/version tracking), professional PDF report generation, a PDF-to-image/text/HTML conversion service, favorites export/import (JSON/CSV), and cloud sync to S3, Google

Cloud Storage, or local folders. Its architecture is deliberately modular: all generic functionality is extracted into 21 reusable `digital.vasic.*` Go submodules plus TypeScript client packages, each independently tested and versioned. QA is driven by the Challenges framework and HelixQA, enforcing anti-bluff, evidence-backed verification.

Why we built it: Existing media managers assume a single storage backend and a single media type. Real collections live across many NAS shares and protocols, degrade when a share drops, and mix movies, music, games, and software. Catalogizer was built to treat all protocols equally, survive flaky network storage, and provide one authoritative, enriched, encrypted catalog over everything.

Why it's a game-changer: It combines protocol-agnostic ingestion, resilient offline-tolerant monitoring, and rich external enrichment in one self-hostable, encrypted package — capabilities normally split across several tools. The extracted-module architecture means the same battle-tested pieces (auth, filesystem, streaming, observability) power an entire product family.

What's innovative: - Unified multi-protocol filesystem client (SMB/FTP/NFS/WebDAV/local) behind one interface. - Offline cache + circuit breaker so the catalog stays usable during storage outages. - Full extraction into 21 reusable `digital.vasic.*` Go submodules and TS client modules. - Encrypted-at-rest catalog (SQLCipher) with real-time WebSocket sync to the UI. - Evidence-backed QA via the Challenges framework and HelixQA integration.

Biggest technical challenges + how solved: - *Flaky network storage:* solved with exponential backoff, circuit breaker, health checks, and an eviction-policy offline cache that serves cached metadata when sources are unreachable. - *Protocol heterogeneity:* solved by abstracting every protocol behind a common `digital.vasic.filesystem` client so higher layers are protocol-unaware. - *Data security:* solved with SQLCipher encryption at rest plus JWT/RBAC auth and request sanitization middleware. - *Maintainability at scale:* solved by extracting all generic logic into independently tested submodules rather than a monolith.

Tech stack (why + how): - **Go + Gin** — high-performance REST API core (`catalog-api`); chosen for concurrency and throughput on continuous monitoring workloads. - **TypeScript + React + Tailwind (Vite)** — responsive catalog-web UI with real-time updates. - **WebSockets** — live data synchronization between backend hub and UI. - **SQLCipher (encrypted SQLite)** — encrypted-at-rest metadata store; dual SQLite/PostgreSQL support via `digital.vasic.database`. - **SMB/FTP/NFS/WebDAV clients** — multi-protocol ingestion via `digital.vasic.filesystem`. -

External metadata APIs (TMDB, IMDB, TVDB, MusicBrainz, Spotify, Steam) — provider plugins for enrichment. - **Prometheus + OpenTelemetry** — metrics/tracing via `digital.vasic.observability`. - **Docker / builder container** — reproducible builds (Tauri/Rust routed through `catalogizer-builder`). - **Redis** — caching/rate limiting via `digital.vasic.cache / ratelimiter`. - **S3 / Google Cloud Storage** — cloud sync and checkpoint storage.

Public links: - GitHub: <https://github.com/vasic-digital/>
Catalogizer (public) - Supporting public repos: `Media-Types-TS`, `Catalogizer-API-Client-TS`, and the React component packages (`Media-Player-React`, `Media-Browser-React`, `Collection-Manager-React`, `Dashboard-Analytics-React`, `Auth-Context-React`) — all public under `vasic-digital`. - Local working copy: `/Volumes/T7/Projects/catalogizer` (has private `.env`; QA reports/.remember are local-only, PRIVATE).

Suggested diagrams/illustrations (OpenDesign): 1. Layered architecture: React UI ↔ Go API ↔ SQLCipher, with the multi-protocol filesystem fan-out to SMB/FTP/NFS/WebDAV/local. 2. Resilience sequence: SMB outage → circuit breaker opens → offline cache serves → reconnect with backoff. 3. Enrichment pipeline: detected file → media classifier → external providers → enriched catalog entry. 4. Module map: Catalogizer app on top of the 21 `digital.vasic.*` submodules.

Site relevance: Primarily `vasic.digital` (company portfolio, flagship product-grade utility). Also suitable as a portfolio highlight on `milosvasic.ru`.

Priority tier: `vasic-util-secondary`

Source provenance: Local repo `/Volumes/T7/Projects/catalogizer/README.md` (features, architecture diagram, 21-module table, resilience section, env vars); `_analysis/github-vasic-digital.md` (repo metadata, description, size); `Courses-Creator.langs.json/_analysis/top20` for related patterns; module list cross-checked against `github-vasic-digital.md`.